VoiceMap — Complete 3-Engineer Build Plan

Los Altos Hacks X · April 11–12, 2026

READ THIS FIRST — ALL ENGINEERS

Before anyone writes a single line of code, everyone reads this section completely. It takes 5 minutes and prevents 5 hours of merge conflicts.

The Golden Rule

You never touch another engineer's files. Ever. If you need something from another engineer's module, you call their API endpoint or import from the shared contract file. That's it. The boundary between engineers is a contract, not a conversation.

The Three Domains

Engineer	Domain	One-line summary
E1	Frontend	Everything the user sees and touches

E2	AI + Voice	OpenAI, ElevenLabs, all prompt logic
E3	Data + Storage	MongoDB, caching, analytics, shortcuts

These domains never overlap. If you find yourself writing code that belongs to another domain, stop and ask.

The Shared Contract File

The single most important file in the project is `/shared/api-contract.ts`. This file is written **once, at the start, by all three engineers together** in the first 30 minutes. It defines every API endpoint, every request shape, every response shape, and every mock response. Once written, no one edits it without telling everyone else.

This file is what allows all three engineers to work completely independently. E1 imports mocks from it. E2 implements endpoints matching it. E3 implements data layers matching it.

The Repository Structure

```
voicemap/
├── shared/
│   ├── api-contract.ts      ← written together, never solo-edited
│   └── mock-data.ts         ← all demo seed data lives here
├── frontend/                ← E1 owns everything in here
│   ├── app/
│   ├── components/
│   ├── utils/
│   └── package.json
├── backend-ai/              ← E2 owns everything in here
│   ├── routes/
```

- │   ├── services/
- │   ├── main.py
- │   └── requirements.txt
- ├── backend-data/ ← E3 owns everything in here
- │   ├── routes/
- │   ├── services/
- │   ├── main.py
- │   └── requirements.txt
- └── .env.example ← lists all required env vars, no values

E1 never opens `backend-ai/` or `backend-data/`. E2 never opens `frontend/` or `backend-data/`. E3 never opens `frontend/` or `backend-ai/`. Violating this causes merge conflicts and confusion. There are no exceptions.

Ports

- Frontend: localhost:3000
- Backend AI: localhost:8001
- Backend Data: localhost:8002

These never change. E1 hardcodes these in a `.env.local` file. If a port isn't running, E1 falls back to mocks automatically — this is built into the frontend from hour one.

THE API CONTRACT

Written together in the first 30 minutes

Every engineer must agree on these before splitting up. This is the entire communication surface between the three domains.

Endpoints E1 calls → E2 (Backend AI, port 8001)

- POST /api/intent
- Request: { path: string[], user_id: string }
- Response: { sentence: string, confidence: number, session_id: string }
- Streams via SSE — tokens arrive one by one
-
- POST /api/confirm
- Request: { session_id: string, user_id: string }
- Response: { audio_url: string, post_session_question: { question: string, question_id: string } | null }
-
- POST /api/feedback
- Request: { session_id: string, thumbs_up: boolean, correction?: string, user_id: string }
- Response: { success: boolean }
-
- POST /api/clarify
- Request: { path: string[], user_id: string }
- Response: { options: { label: string, icon: string, path: string[] }[] }
-
- POST /api/caregiver/simplify
- Request: { text: string, user_id: string }
- Response: { simplified: string[], emoji_tags: string[], audio_url: string }
-
- GET /api/digest
- Params: { user_id: string }
- Response: { digest: string }

Endpoints E1 calls → E3 (Backend Data, port 8002)

- GET /api/profile
- Params: { user_id: string }
- Response: { profile, medical, preferences, routine, voice_id, interface_settings }
-
- POST /api/profile/update
- Request: { user_id: string, field: string, value: any }
- Response: { success: boolean, knowledge_score: number }
-
- GET /api/shortcuts
- Params: { user_id: string }
- Response: { shortcuts: { path: string[], label: string, icon: string, tap_count: number }[] }
-
- GET /api/predictions
- Params: { user_id: string, hour: number }
- Response: { predictions: { label: string, path: string[], icon: string }[] }
-
- POST /api/context/answer
- Request: { user_id: string, question_id: string, question: string, answer: string }
- Response: { success: boolean, knowledge_score: number }
-
- GET /api/caregiver/panel
- Params: { user_id: string }
- Response: { last_session, pending_question, knowledge_score, knowledge_breakdown, urgent: boolean }
-
- GET /api/sessions/history
- Params: { user_id: string, limit: number }
- Response: { sessions: Session[] }
-
- GET /api/insights

- Params: { user_id: string }
- Response: { sessions_by_day, top_paths, sessions_by_period, mood_log }
-
- POST /api/sessions/flag
- Request: { session_id: string, user_id: string }
- Response: { success: boolean }
-
- GET /api/buttons/custom
- Params: { user_id: string }
- Response: { buttons: { id, label, icon, category }[] }
-
- POST /api/buttons/add
- Request: { user_id: string, label: string, icon: string, category: string }
- Response: { success: boolean, button_id: string }

Shared TypeScript types (in `/shared/api-contract.ts`)

- type Session = {
- id: string
- path: string[]
- sentence: string
- confidence: number
- audio_url: string
- feedback: 'positive' | 'correction' | null
- correction: string | null
- is_first_occurrence: boolean
- timestamp: string
- }
-
- type KnowledgeBreakdown = {
- overall: number
- profile: number

- medical: number
- preferences: number
- conversation: number
- }
-
- type CaregiverPanel = {
- last_session: Session | null
- pending_question: { question: string, question_id: string } | null
- knowledge_score: number
- knowledge_breakdown: KnowledgeBreakdown
- urgent: boolean
- }

MOCK DATA

`/shared/mock-data.ts` — used by E1 when backends aren't running

Every mock response exactly mirrors the real API contract shape. E1 never writes her own mock data — she imports from here. E2 and E3 reference this to understand what their endpoints must return.

- export const MOCK_USER_ID = "alex_demo"
-
- export const MOCK_PROFILE = {
- profile: { name: "Alex", diagnosis_date: "2024-04-12", caregiver_name: "Maya" },
- medical: { medications: ["Aspirin 100mg", "Lisinopril 10mg"], allergies: ["Penicillin"], conditions: ["Hypertension"] },
- preferences: { communication_notes: "Alex gets frustrated when misunderstood. Give him time.", known_preferences: "Loves Italian

food, especially tiramisu. Watches football on Sundays.",
always_know: "His daughter Maria lives in Boston. He misses her." },

- routine: { meals: { breakfast: "08:00", lunch: "12:30", dinner: "18:00" } },
- voice_id: "mock_voice_id",
- interface_settings: { simplified_view: false, show_subtitles: true, shortcut_threshold: 5 }
- }
-
- export const MOCK_SHORTCUTS = {
- shortcuts: [
- { path: ["food","dessert","tiramisu"], label: "Tiramisu", icon: "bowl-food", tap_count: 14 },
- { path: ["needs","medicine"], label: "Medicine", icon: "pill", tap_count: 11 },
- { path: ["feelings","tired"], label: "Tired", icon: "moon", tap_count: 8 },
- { path: ["needs","bathroom"], label: "Bathroom", icon: "drop", tap_count: 7 },
- { path: ["people","family"], label: "Family", icon: "users", tap_count: 6 },
-]
- }
-
- export const MOCK_INTENT_RESPONSE = {
- sentence: "I want tiramisu — it's my favorite.",
- confidence: 0.91,
- session_id: "mock_session_001"
- }
-
- export const MOCK_CONFIRM_RESPONSE = {
- audio_url: "/mock-audio/tiramisu.mp3",
- post_session_question: {
- question: "Does Alex prefer a specific dessert? I want to get more specific next time.",

```
    question_id: "q_dessert_pref"
  }
}

export const MOCK_CAREGIVER_PANEL: CaregiverPanel = {
  last_session: {
    id: "mock_session_001",
    path: ["food","dessert","tiramisu"],
    sentence: "I want tiramisu — it's my favorite.",
    confidence: 0.91,
    audio_url: "/mock-audio/tiramisu.mp3",
    feedback: "positive",
    correction: null,
    is_first_occurrence: true,
    timestamp: new Date().toISOString()
  },
  pending_question: {
    question: "Does Alex prefer a specific dessert?",
    question_id: "q_dessert_pref"
  },
  knowledge_score: 71,
  knowledge_breakdown: { overall: 71, profile: 94, medical: 78, preferences: 61, conversation: 42 },
  urgent: false
}

export const MOCK_PREDICTIONS = {
  predictions: [
    { label: "I need my medication", path: ["needs","medicine"], icon: "pill" },
    { label: "Good morning", path: ["feelings","happy"], icon: "sun" },
    { label: "I'm hungry", path: ["food"], icon: "fork-knife" },
  ]
}

```

```
export const MOCK_INSIGHTS = {
  sessions_by_day: { /* 30 days of data */ },
  top_paths: [
    { path: "food → dessert → tiramisu", count: 14 },
    { path: "needs → medicine", count: 11 },
    { path: "feelings → tired", count: 8 },
  ],
  sessions_by_period: { morning: 34, afternoon: 28, evening: 19 },
  mood_log: [
    { date: "2025-04-08", score: 7, notes: "Good day" },
    { date: "2025-04-07", score: 5, notes: "Tired" },
    { date: "2025-04-06", score: 8, notes: "Very engaged" },
  ]
}
```

ENGINEER 1 — FRONTEND

You own: `/frontend/` entirely

You never touch: `/backend-ai/`, `/backend-data/`

Your Mindset

You are building a product that judges interact with directly. Every second of confusion, every loading state that feels broken, every layout that shifts unexpectedly — judges feel all of it. Your job is to make every interaction feel intentional and polished. The backends are not your concern. If they're not ready, you use mocks. You should never be blocked waiting for E2 or E3.

Your API Layer — The Key to Independence

The single most important thing you build is `frontend/utils/api.ts`. Every API call in the entire frontend goes through this file. It has one job: try the real backend, fall back to mock data if it fails.

```
// Every API function follows this pattern
export const getShortcuts = async (userId: string) => {
  try {
    const res = await fetch(`${AI_BASE}/api/shortcuts?user_id=${userId}`)
    if (!res.ok) throw new Error()
    return await res.json()
  } catch {
    console.warn('Backend unavailable — using mock data')
    return MOCK_SHORTCUTS
  }
}
```

This means: from minute one, every screen in your app works. No blank screens. No errors. No waiting for E2 or E3 to finish anything. You build, test, and demo with mocks. When backends are ready, real data flows in automatically without changing a single line of your code.

Tech Stack

- Next.js 14 with App Router
- TypeScript throughout
- Tailwind CSS for styling
- Recharts for all data visualizations
- Phosphor Icons for all patient-facing icons (`@phosphor-icons/react`)
- Framer Motion for animations

Screen Architecture

- / → Landing / demo profile loader
- /onboarding → 5-step caregiver setup flow
- /patient → main communication screen
- /caregiver → full dashboard

The Layout System — Build This First

The patient screen is a two-column resizable layout. This is the foundation everything else sits on. Build it before any other component.

The layout has a drag handle between the left column (patient area) and right column (caregiver panel). The panel ranges from fully closed to exactly 50% width. Below 5% it snaps fully closed. Default: 70-30.

Store the split ratio in React state. Persist it to localStorage so it survives refreshes. The drag handle should feel smooth — use `pointer` events, not `mouse` events, so it works on touch screens too.

Once this layout renders correctly with placeholder content in both columns, move on. Don't perfect it now — you'll tune it in the polish pass.

Patient Area — Left Column

Build components in this exact order. Each one is independently testable with mock data.

1. PredictionBar

Thin row at the very top. On mount, calls `GET /api/predictions` with the current hour. Renders 3-4 clickable phrase chips. Tapping a chip fires the same flow as tapping a leaf node — straight to output, skipping the grid.

If the backend isn't ready, render the mock predictions from `MOCK_PREDICTIONS`. The component should look identical either way.

2. ModeToggle

Two tabs: **Tree** and **Composer**. Stores active mode in React state. Tapping switches the grid below between `TreeGrid` and `ComposerGrid`. No backend dependency.

3. ShortcutBar

Loads from `GET /api/shortcuts`. Renders starred shortcut chips above the main grid. On app load, fire `prefetchShortcutAudio()` — a background function that calls `POST /api/confirm` for each shortcut path and stores the returned `audio_url` in localStorage. When a shortcut is tapped later, check localStorage first before making any API call.

```
const prefetchShortcutAudio = async (shortcuts) => {
  await Promise.all(shortcuts.map(async (s) => {
    const key = `audio_${s.path.join('_')}`
    if (localStorage.getItem(key)) return // already cached
    const result = await confirmPath(s.path, userId)
    if (result?.audio_url) {
      localStorage.setItem(key, result.audio_url)
    }
  }))
}
```

4. TreeGrid

The main button grid in Tree mode. Renders the current level of the decision tree as a 2-column grid of large icon cards. Each card: Phosphor icon (large, ~70% of card height), label text, small subtitle (only if `show_subtitles` is true in user settings).

The decision tree structure is hardcoded in `frontend/utils/tree-data.ts`. This file never changes and has no backend dependency. It's a pure data file.

Tree data structure (abbreviated):

```
export const TREE = {
  food: { label: "Food", icon: "fork-knife", children: {
    dessert: { label: "Dessert", icon: "cake", children: {
      tiramisu: { label: "Tiramisu", icon: "bowl-food", leaf: true, subtitle: "Italian dessert" },
      icecream: { label: "Ice Cream", icon: "ice-cream", leaf: true, subtitle: "Cold dessert" },
    }},
    drink: { label: "Drink", icon: "cup", children: {
      water: { label: "Water", icon: "drop", leaf: true },
      tea: { label: "Tea", icon: "thermometer", leaf: true },
    }},
  }},
  feelings: { label: "Feelings", icon: "smiley", children: { ... }},
  needs: { label: "Needs", icon: "hand-waving", children: { ... }},
  people: { label: "People", icon: "users", children: { ... }},
  health: { label: "Health", icon: "heart", children: { ... }},
}
```

Tapping a non-leaf node drills down — update current path in state, re-render grid with children. Tapping a leaf node triggers the intent flow. A breadcrumb trail above the grid shows the current path. A back button is always visible once drilled in.

5. ComposerGrid

The icon grid in Composer mode. A flat scrollable grid of all icons organized into horizontal labeled groups: Actions · Objects · Places · Times · Feelings · People · Medical.

Tapping an icon adds it to the SentenceTray below. No drilling, no hierarchy. Just icons.

Fitts' Law scaling: When icons are in the tray, query `path_frequencies` from the loaded shortcuts data to find which icons statistically follow the current sequence. Icons that are likely next candidates scale up by 15% and get a subtle highlight border. This is a CSS transform — no backend call required. Implement after core composer flow works.

6. SentenceTray

Persistent tray at the bottom of the patient area, only visible in Composer mode. Shows tapped icons left to right as removable chips. Clear button. Confirm button.

When Confirm is tapped, the array of icon names becomes the `path` sent to `POST /api/intent`. The rest of the flow is identical to the tree flow.

7. OutputPanel

Appears after any confirmation — tree leaf, composer confirm, or shortcut chip. This is the most important component in the app.

States in order:

State 1 — Generating (immediate on confirm tap): Show a pulsing waveform animation immediately. Five bars, CSS keyframe animation, plays before any API response. This is purely visual — no data needed. It signals "something is happening" within 50ms of the tap.

State 2 — Streaming: As SSE tokens arrive from `POST /api/intent`, words appear one by one in the output panel alongside the waveform. The sentence builds visibly. First token typically arrives in 300-500ms.

State 3 — Generating audio: Sentence is complete. `POST /api/confirm` is called. Waveform transitions to a "synthesizing" state — slightly different animation to show the voice is being prepared.

State 4 — Playing: Audio URL returned. Create an `Audio` object and play it. Waveform transitions to a playback animation — bars move in sync with the audio. Show label: *"Speaking in Alex's voice."*

State 5 — Complete: Audio finished. Show confirmation. Display the first-time badge if `is_first_occurrence` is true from the session data.

Confidence handling:

- `>75%` → proceed through all 5 states normally
- `50-75%` → same flow, but show amber dot, caregiver panel pulses
- `<50%` → skip states 3-5, instead call `POST /api/clarify` and render `ClarificationButtons`

ClarificationButtons: 2-3 large tappable cards with icon + short label. Tapping one fires the full intent flow with the specific clarified path. No text reading required.

localStorage fallback: Before calling `POST /api/intent`, check `localStorage.getItem(`sentence_${path.join('_')}`)`. If found, skip the API call entirely and jump to State 4. After every successful generation, write `{ sentence, audio_url }` to localStorage with the path as key.

Caregiver Panel — Right Column

Five sections stacked vertically. All data comes from `GET /api/caregiver/panel` polled every 3 seconds.

LiveActivity

Top section. Shows:

- Current mode badge: "Tree" or "Composer"
- Live breadcrumb of what's currently selected in the patient area — this is local React state shared between columns, not an API call
- Streaming sentence preview — the same token stream that the OutputPanel receives, mirrored here
- Confidence dot for last generated sentence
- Small label at very top: *"Caregiver view — Alex sees icons only"*

The breadcrumb is local state — it updates instantly as the patient taps because it lives in the same React app. No polling delay.

SessionHistory

Scrollable list of confirmed sessions from `GET /api/sessions/history`. Most recent first.

Each row:

- Timestamp and sentence
- Confidence dot
- ⭐ badge if `is_first_occurrence`
- Thumbs up / thumbs down — tapping thumbs down reveals an inline text input
- On correction submit → `POST /api/feedback` with `thumbs_up: false` and the correction text
- ▸ replay button — stores `audio_url` per session, plays it directly from the URL. Zero API call.
- Flag button → `POST /api/sessions/flag`

KnowledgeScore

The visual proof that the system is learning.

Main percentage bar at top. Four mini-bars below:

- Profile 94%
- Medical 78%
- Preferences 61%
- Conversation 42% ← needs attention

Below the bars: *"Answering the question below will improve Preferences by ~3%"* — this is calculated from the knowledge breakdown data returned by the API.

When the caregiver answers a question, animate the relevant mini-bar filling slightly. This is what you run in the demo to show the system learning in real time.

ClaudeQuestion

Rendered as a highlighted card. Shows the `pending_question` from the caregiver panel API response.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ 💬 Does Alex prefer a specific dessert?                                      │
│    I want to get more specific next time.                                    │
│                                                                              │
│ [text input]                                                  [Send] [Skip]  │
│                                "Add to profile instead →"                    │
└──────────────────────────────────────────────────────────────────────────────┘
```

Submitting → `POST /api/context/answer`. On success: knowledge score animates up, relevant mini-bar fills, card shows "Got it ✓" briefly then clears.

"Add to profile instead →" opens the relevant profile section in a modal overlay — same form fields as the full Profile tab, scoped to just the relevant category. No navigation required.

UrgencyAlert

Conditionally rendered based on `urgent: true` in the caregiver panel response. When urgent:

- Panel border turns persistent red
- Alert card pins to top of panel above all other sections
- *"Alex has expressed pain or distress 3 times in the last 2 hours."*
- Dismiss button — calls `POST /api/sessions/flag` on the triggering sessions

Full Caregiver Dashboard `/caregiver`

Three tabs. Opened via "Full Dashboard →" button at the bottom of the caregiver panel.

Profile Tab

All patient context editable in place. Organized into four sections matching the four knowledge sub-scores:

Patient Profile: Name, photo, diagnosis date, caregiver relationship. Edits save on blur via `POST /api/profile/update`.

Medical: Medications as tag chips (type + enter = chip), allergies, conditions. Same save-on-blur pattern.

Preferences: Communication notes (textarea), known preferences (textarea), "Always remember" (textarea). Character count shown — context is fed to OpenAI and length matters.

Custom Buttons: List of caregiver-added buttons. Each shows Phosphor icon + label + category. Add button opens a small form: text input for label, Phosphor icon picker (searchable grid of icons), category dropdown. Submit → `POST /api/buttons/add`. New button appears immediately in the patient's grid in the relevant category.

Daily Routine: Visual timeline — morning block, afternoon block, evening block. Each block has editable medication times, meal times. Saves on blur.

Insights Tab

Four Recharts components. All data from `GET /api/insights`.

Sessions per day: Bar chart, last 14 days. Color bands by time of day within each bar (morning = teal, afternoon = amber, evening = navy). Shows clear growth trend from the seeded data.

Top communication paths: Horizontal bar chart, top 8 paths. Judges can see at a glance what Alex communicates most.

Time-of-day breakdown: Simple three-bar chart or donut. Morning / Afternoon / Evening.

Mood trend: Line chart. Caregiver-logged score 1-10 over 14 days. Overlaid with session frequency line — correlation visible.

Daily digest card pinned to top of tab. Claude-generated 2-3 sentence summary. If no digest for today, show yesterday's with a timestamp. Tapping "Regenerate" calls `GET /api/digest`.

History Tab

Full paginated session log. Columns: timestamp, path taken, sentence, confidence, feedback status, first occurrence badge, flag status.

Filterable by: date range (date pickers), category (dropdown from top-level tree categories), feedback status (all / positive / corrected / flagged).

"Share with Doctor" button at top right. Generates a PDF using jsPDF with: patient name, date range, top communication patterns, mood trend summary, flagged sessions. Judges see this as the "real product" moment — this isn't a toy, it's a clinical tool.

Onboarding `/onboarding`

5 steps. Progress bar at top. Back button on all steps except Step 1.

Step 1 — Welcome: Product explanation, 3 bullet points. Two prominent buttons:

- "Set up for [patient name]" → starts the full flow
- "Load Demo Profile" → calls a single endpoint `POST /api/demo/seed`, waits for confirmation, redirects to `/patient`. **Build this button first. Test it first. It is your safety net.**

Step 2 — Patient Basics: Name (text), photo (file upload, optional), diagnosis date (date picker), caregiver relationship (dropdown).

Step 3 — Medical Context: Medications (tag input), allergies (tag input), other conditions (tag input), doctor name (optional).

Step 4 — Communication Notes + Preferences: Three textareas: what they struggle to express, known preferences, always remember. Character count shown on each.

Step 5 — Voice Recording: Large centered microphone button. Script displayed for patient to read. `MediaRecorder` API captures audio.

Progress bar during recording (60 seconds). "Upload & Clone" button calls `POST /api/voice/clone` on E2. Status states: Idle → Recording → Processing → "Voice saved ✓." Skip button available — falls back to preset warm voice. After voice is saved, background job pre-generates sentences for all leaf paths.

What You Test At Each Stage

Hours 1-3: Layout renders correctly with drag handle. Both columns visible. Mock data showing in caregiver panel. Button grid renders with Phosphor icons.

Hours 3-7: Full tree navigation works with mocks. Leaf tap triggers waveform animation. Mock sentence appears. Mock audio plays (use a local MP3 file as the mock audio URL).

Hours 7-11: Composer mode works. Icons snap into tray. Confirm fires the same flow. Caregiver panel updates. Thumbs up/down work against mock data.

Hours 11-15: Real API data replaces mocks when backends are ready. Behavior should be identical — just real sentences and real audio.

Hours 15-18: Polish. Animations. Charts. Insights tab. Onboarding flow.

Do Not Touch

- `/backend-ai/` — any file in here
- `/backend-data/` — any file in here
- `/shared/api-contract.ts` — only edit with everyone present
- `.env` — you have `.env.local` for your own values

ENGINEER 2 — BACKEND AI

You own: `/backend-ai/` entirely

You never touch: `/frontend/`, `/backend-data/`

Your Mindset

You are the intelligence layer. Every sentence Alex speaks, every question Claude asks Maya, every daily summary — that's yours. Your job is to make the AI feel genuinely personalized and fast. The most important thing you'll build is `build_context_string()` — the function that turns everything stored about Alex into a prompt that makes GPT-4o-mini sound like it knows him personally.

You never need the frontend to be ready. You test every endpoint with curl or a simple test script. You never need E3's database to be ready — you have a mock data module that simulates every data response you'd normally get from E3.

Your Mock Data Module

`/backend-ai/mock_data.py` — mirrors exactly what E3's database functions return. You import from this whenever E3's functions aren't available.

- MOCK_USER = {
- "profile": {"name": "Alex"},
- "medical": {"medications": ["Aspirin 100mg", "Lisinopril 10mg"], "allergies": ["Penicillin"]},
- "preferences": {
- "communication_notes": "Alex gets frustrated when misunderstood.",

- "known_preferences": "Loves tiramisu. Watches football.",
- "always_know": "His daughter Maria lives in Boston."
- },
- "correction_history": [
- {"path": "food → dessert → cake", "original": "I would like cake.", "corrected": "I want tiramisu specifically."}
-],
- "context_answers": [
- {"question": "Favorite dessert?", "answer": "Tiramisu, always tiramisu"},
- {"question": "Favorite sport?", "answer": "Football, 49ers fan"}
-],
- "voice_id": "mock_voice_id"
- }

Every function that normally calls E3's database has a fallback to this mock. When E3 is ready, you swap the mock for the real database call with a single flag.

Tech Stack

- Python 3.11+
- FastAPI
- openai (Python SDK)
- elevenlabs (Python SDK)
- python-dotenv
- uvicorn

The Context Assembly Function

This is the most important function you write. It assembles everything known about the patient into a dense, capped system prompt. Every token

here has to earn its place — you're capped at 300 tokens for the system prompt because longer prompts mean slower responses.

Priority order:

1. Correction history (most impactful — direct behavioral feedback)
2. Context answers from caregiver questions (learned preferences)
3. Known preferences and communication notes
4. Medical context (medications, allergies)
5. Basic profile (name, relationship)

Hard limits: last 10 corrections only, last 15 context answers only, truncate preferences to 150 chars if needed. The function always produces a prompt under 300 tokens regardless of how much data is in the database.

The output looks like:

- Patient: Alex, aphasia post-stroke.
- Meds: Aspirin 100mg, Lisinopril 10mg. Allergies: Penicillin.
- Preferences: Loves tiramisu. Football fan (49ers). Daughter Maria in Boston.
- Notes: Gets frustrated when misunderstood. Give him time.
- Corrections: food→dessert→cake = "I want tiramisu specifically, not just cake"
- Learned: Favorite dessert = tiramisu | Morning mood = slow to wake

Endpoints

POST /api/intent — Streaming sentence generation

This is your most performance-critical endpoint. It must feel fast.

Flow:

1. Get user data (from E3 if available, from mock if not)
2. Build context string (capped at 300 tokens)

3. Call GPT-4o-mini with streaming enabled
4. Return an SSE stream — tokens flow to the frontend one by one
5. After full sentence is assembled, compute confidence score
6. Store pending session data by calling E3's `POST /api/sessions/create` (or mock it)

The confidence score is computed by calling GPT-4o-mini a second time with a simple prompt: *"Rate how confident you are about this sentence given this context. 0.0 to 1.0. Return only a number."* This second call runs after the stream completes so it doesn't add to perceived latency.

Response via SSE:

- data: {"token": "I "}
- data: {"token": "want "}
- data: {"token": "tiramisu"}
- data: {"token": " — "}
- data: {"token": "it's "}
- data: {"token": "my "}
- data: {"token": "favorite."}
- data: {"done": true, "session_id": "s_abc123", "confidence": 0.91}

POST /api/confirm — ElevenLabs synthesis

Called after the patient confirms the streamed sentence.

Flow:

1. Get session data from E3 (sentence, user_id)
2. Get user's voice_id from E3 profile
3. Call ElevenLabs API — `eleven_multilingual_v2` for patient voice (quality)
4. Save audio file to `/tmp/voicemap_audio/`
5. Trigger post-session question generation as a background task
6. Return audio URL

The post-session question runs in a background task so it never blocks the audio response:

- @router.post("/api/confirm")
- async def confirm(req, background_tasks: BackgroundTasks):
- audio_url = await synthesize(sentence, voice_id)
- background_tasks.add_task(generate_and_save_question, session_id, user_id)
- return {"audio_url": audio_url}

POST /api/feedback — Learn from corrections

When a caregiver submits a correction:

1. Get the original session (path + sentence)
2. If correction provided: call GPT-4o-mini to generate an improved sentence incorporating the correction
3. Call E3 to save the correction to the user's correction history
4. Call E3 to overwrite the cached sentence for that path

This is what closes the learning loop. The next time that path is tapped, the cached sentence reflects the correction.

POST /api/clarify — Low confidence alternatives

Called when confidence is below 0.5.

Takes the ambiguous path and generates 2-3 specific clarification options. Each option has a label, an icon name, and a specific path that, if tapped, will produce a high-confidence sentence.

Uses GPT-4o-mini with a structured JSON response prompt. Returns within 500ms.

POST /api/caregiver/simplify — Caregiver-to-patient communication

Takes a complex sentence the caregiver wants to say to Alex.

Two steps:

1. GPT-4o-mini simplifies to 1-2 short sentences (max 6 words each)
2. ElevenLabs synthesizes using `eleven_turbo_v2` and a neutral preset voice (not Alex's cloned voice — this is the caregiver speaking TO Alex)

The voice distinction is intentional and explicit. Patient output = Alex's cloned voice labeled *"Alex's voice."* Caregiver output = neutral voice labeled *"Caregiver voice."* This is your ElevenLabs prize moment.

GET /api/digest — Daily summary

Called on first caregiver dashboard open of the day, or on manual refresh.

Gets last 24 hours of sessions from E3. Passes session data to GPT-4o (full model, quality matters here). Generates 2-3 warm, specific sentences. Flags any first-occurrence sessions with a star.

Example output: *"Alex communicated 11 times today. His most frequent need was food, particularly asking for tiramisu twice. Mood seemed calm based on session frequency and timing. Notable: first time he expressed 'I love you' using the People category ⭐."*

POST /api/voice/clone — Onboarding voice cloning

Takes audio file from onboarding. Calls ElevenLabs Instant Voice Cloning API. Saves voice_id to user profile via E3. Returns voice_id.

Before the hackathon: Record a clean 60-second audio sample, run this endpoint, save the voice_id to `.env` as `ALEX_VOICE_ID`. The demo mode seed script uses this pre-cloned ID so voice cloning never needs to happen live during the demo.

POST /api/demo/seed — Demo mode trigger

Called when E1's "Load Demo Profile" button is pressed. Calls E3's seed endpoint to populate MongoDB with Alex's full profile and 30 days of

sessions. Returns when seeding is complete. E1 then redirects to `/patient`.

Your Testing Approach

You never need E1 or E3 to test. Every endpoint is testable with curl:

- # Test intent streaming
- curl -X POST localhost:8001/api/intent \
- -H "Content-Type: application/json" \
- -d '{"path": ["food","dessert","tiramisu"], "user_id": "alex_demo"}'
-
- # Test confirm
- curl -X POST localhost:8001/api/confirm \
- -d '{"session_id": "test_001", "user_id": "alex_demo"}'

Write a `test_endpoints.py` script that tests every endpoint in sequence. Run it after every significant change.

Latency Targets

Endpoint	Target
/api/intent first token	< 400ms

/api/intent full sentence	< 1.5s
/api/confirm (ElevenLabs)	< 2s
/api/clarify	< 600ms
/api/caregiver/simplify	< 2s

If you're exceeding these, the primary lever is shortening the context string. Every 100 tokens removed from the system prompt saves ~200ms.

Do Not Touch

- `/frontend/` — any file in here
- `/backend-data/` — any file in here
- `/shared/api-contract.ts` — only edit with everyone present
- E3's database functions — call them via HTTP, never import them directly

---

ENGINEER 3 — BACKEND DATA

You own: `/backend-data/` entirely

You never touch: `/frontend/`, `/backend-ai/`

Your Mindset

You are the memory layer. Everything that makes VoiceMap feel like it knows Alex — the sentence cache, the knowledge score, the shortcut engine, the analytics — lives in your code. The demo's most impressive moment (knowledge score ticking up in real time) depends entirely on your knowledge score engine working correctly.

You never need E1 or E2 to be ready. You test with curl and a dedicated test script. Your seed script is the first thing you run and the most important thing you build — everyone depends on it.

Tech Stack

- Python 3.11+
- FastAPI
- motor (async MongoDB driver)
- pymongo
- python-dotenv
- uvicorn

MongoDB Collections

Five collections. Every collection has a clear owner and purpose.

`users` — One document per user. Contains everything known about the patient. This is the single source of truth for all personalization.

Fields: `_id`, `name`, `voice_id`, `profile`, `medical`, `preferences`, `routine`, `interface_settings`, `custom_buttons`, `correction_history`, `context_answers`, `knowledge_score`, `knowledge_breakdown`, `path_frequencies`, `mood_log`

`sessions` — One document per communication session. Every time Alex confirms a sentence, a session is created.

Fields: `_id`, `user_id`, `path`, `sentence`, `confidence`, `audio_url`, `feedback`, `correction`, `is_first_occurrence`, `flagged`, `timestamp`

`sentences` — The sentence cache. One document per user + path combination. Pre-generated sentences live here. This is what makes repeat taps instant.

Fields: `_id` (format: `{user_id}_{path_key}`), `user_id`, `path_key`, `sentence`, `confidence`, `audio_url`, `personalized`, `last_updated`

`context_log` — Every question Claude asked and every answer the caregiver gave. Fed back into E2's context assembly function.

Fields: `_id`, `user_id`, `question_id`, `question`, `answer`, `timestamp`

`anchors` — Generated anchor audio sessions (pre-sleep audio). Not used in core demo but schema is ready.

The Seed Script — Run This First

`/backend-data/seed_demo.py` is the most important file you write. Run it before doing anything else. It populates MongoDB with everything the app needs to function.

What it seeds:

Alex's full user document — complete profile with all medical data, preferences, corrections, context answers, pre-set knowledge score of 71, path frequencies for shortcuts.

30 days of sessions — structured with a clear narrative arc:

- Days 1-10: 2-3 sessions/day, confidence 0.55-0.70, frequent corrections, `anchor_used: false`
- Days 11-20: 3-4 sessions/day, confidence 0.70-0.82, fewer corrections, `anchor_used: true` sometimes
- Days 21-30: 4-6 sessions/day, confidence 0.82-0.94, shortcuts appearing, high frequency

This arc makes the bar chart show visible growth. Without it, the chart is flat and random.

Pre-cached sentences for all top-5 shortcut paths — these return instantly from the cache without any API call.

Two urgency-triggering sessions timestamped to today — three pain/help sessions within 2 hours. This makes the urgency alert demonstrable on demand during the demo.

Today's mood log entry — score 7, notes "Good day, communicated well."

Run it with: `python seed_demo.py`. Takes about 5 seconds. Run it again to reset demo data at any time.

The Sentence Cache — Core Latency Solution

The get-or-generate pattern is the fundamental operation of your data layer. Every intent request goes through this.

- Receive path + user_id
- Check sentences collection for {user_id}_{path_key}

- → Found: return cached sentence + audio_url instantly (~50ms)
- → Not found: return None (E2 will generate and then call POST /api/sentences/cache to store it)

E2 calls `POST /api/sentences/cache` after every successful generation. You store it. Next time E1 requests that path, you return it instantly from cache.

When a caregiver submits a correction, E2 calls `POST /api/sentences/invalidate` with the path key. You delete that cache entry. Next tap regenerates with the corrected context.

The Knowledge Score Engine

Runs every time a caregiver answers a question, edits the profile, or a session is confirmed. Returns a score from 0-100 and a breakdown by category.

Four sub-scores, each computed independently:

Profile (25 points max):

- Name present: 5pts
- Diagnosis date: 3pts
- Caregiver name: 3pts
- Photo uploaded: 2pts
- Communication notes filled: 6pts
- Known preferences filled: 6pts

Medical (25 points max):

- At least one medication: 8pts
- Allergies filled: 7pts
- Other conditions: 5pts
- Doctor name: 5pts

Preferences (25 points max):

- Context answers given: 2pts each, capped at 25pts

Conversation (25 points max):

- Sessions confirmed: 1pt each, capped at 15pts
- Corrections given: 2pts each, capped at 10pts

Total is the sum of all four sub-scores, capped at 100. The breakdown object shows each sub-score as a percentage of its maximum. This feeds the four mini-bars in E1's caregiver panel.

The Shortcut Engine

Runs on every `GET /api/shortcuts` call. Reads `path_frequencies` from the user document. Returns any path that has been tapped at least `shortcut_threshold` times (default: 5, configurable per user).

Sorted by frequency descending, capped at 5 shortcuts. Each shortcut includes the path, a human-readable label (last element of path, title-cased), the Phosphor icon name for that leaf node, and the tap count.

After every confirmed session, call `$inc` on `path_frequencies.{path_key}`. When a path crosses the threshold, it automatically appears in the shortcut bar on the next poll.

The Urgency Detection Engine

Runs on every `GET /api/caregiver/panel` call. Simple aggregation:

- two_hours_ago = datetime.utcnow() - timedelta(hours=2)
- distress_count = count sessions where:
- user_id matches AND
- timestamp > two_hours_ago AND
- any element of path is in ["pain", "help", "emergency", "health"]
-

- return distress_count >= 3

Returns `urgent: true` in the panel response when triggered. E1 renders the red alert. No additional infrastructure needed.

The Predictions Engine

`GET /api/predictions` receives the current hour (0-23) from E1. Returns the 3-4 most likely phrases for that time of day based on session history.

- # Get sessions from same time window across all past days
- hour_range = (hour - 1, hour + 1) # ±1 hour window
- relevant_sessions = sessions where timestamp.hour in hour_range
-
- # Rank by frequency
- path_counts = count occurrences of each path in relevant_sessions
- return top 3-4 paths with their labels and icons

For the demo, seed data includes enough morning sessions that "I need my medication" and "I'm hungry" always appear at the top during morning demo hours.

The First Occurrence Detection

Runs before every session is saved:

- existing = find one session where user_id matches AND path matches
- is_first = existing is None

Save `is_first_occurrence` as a boolean on the session document. E1 uses it to render the ⭐ badge.

Endpoints

GET /api/profile

Returns the full user document (minus internal MongoDB fields). Used by E1 to populate the caregiver dashboard profile tab.

POST /api/profile/update

Receives a field path and value. Updates the user document using MongoDB's `$set` operator. Recalculates and returns the updated knowledge score. E1 calls this on every profile field blur event.

GET /api/shortcuts

Runs the shortcut engine. Returns top 5 shortcuts with all display fields.

GET /api/predictions

Runs the prediction engine for the given hour. Returns 3-4 predicted phrases.

POST /api/sessions/create

Called by E2 when a new intent is generated. Creates a pending session document. Returns the new session_id.

POST /api/sessions/confirm

Called by E2 after ElevenLabs synthesis. Updates session with audio_url, marks as confirmed. Increments path frequency. Runs first-occurrence check.

POST /api/feedback

Saves thumbs up/down and optional correction to the session. If correction provided, adds to user's correction_history and calls `POST /api/sentences/invalidate`.

POST /api/context/answer

Saves question + answer to context_log. Also appends to user's context_answers array (capped at most recent 20). Recalculates and returns knowledge score.

GET /api/caregiver/panel

Aggregates everything E1 needs for the right panel in one call: last confirmed session, pending question from last session, knowledge score with breakdown, urgency flag. Designed to be called every 3 seconds — keep it fast.

GET /api/sessions/history

Returns paginated session history for the history tab. Supports filters: date range, category, feedback status.

POST /api/sessions/flag

Marks a session as flagged for doctor review.

GET /api/insights

Aggregates all chart data: sessions_by_day (14 days), top_paths (8 paths), sessions_by_period, mood_log. Called once on dashboard open, not on a poll loop.

GET /api/buttons/custom

Returns caregiver-added custom buttons for the patient grid.

POST /api/buttons/add

Adds a new custom button to the user document. Returns the new button_id.

POST /api/sentences/cache

Called by E2 after every successful generation. Stores sentence + audio_url keyed by `{user_id}_{path_key}`.

POST /api/sentences/invalidate

Called by E2 after a correction. Deletes the cached sentence for a path so it regenerates fresh.

POST /api/demo/seed

Runs the full seed script programmatically. Called by E2 when E1's "Load Demo Profile" button is pressed. Returns when seeding is complete.

Your Testing Approach

Write `test_endpoints.py` that tests every endpoint in sequence using `httpx`. Run it after every change.

```
import httpx

BASE = "http://localhost:8002"

def test_profile():
    r = httpx.get(f"{BASE}/api/profile", params={"user_id": "alex_demo"})
    assert r.status_code == 200
    assert r.json()["profile"]["name"] == "Alex"
    print("✓ profile")

def test_knowledge_score():
    r = httpx.post(f"{BASE}/api/context/answer", json={
        "user_id": "alex_demo",
```

```
        "question_id": "test_q",
        "question": "Favorite food?",
        "answer": "Tiramisu"
    })
    assert r.json()["knowledge_score"] > 71
    print("✓ knowledge score increases on answer")

# Run all tests
test_profile()
test_knowledge_score()
# ... etc
```

Do Not Touch

- `/frontend/` — any file in here
- `/backend-ai/` — any file in here
- `/shared/api-contract.ts` — only edit with everyone present
- E2's services — call them via HTTP if needed, never import directly

---

INTEGRATION GUIDE

How the three domains connect

Integration happens in two phases. Each phase has a clear trigger.

Phase 1 — E3 + E2 Integration (hours 8-10)

E2 switches from mock data to real E3 data. E2's `get_user()` function changes from returning `MOCK_USER` to calling `GET`

`localhost:8002/api/profile`. E2's session creation changes from no-ops to real calls to E3.

Trigger: E3 says "seed script works, all endpoints return 200."

Test: E2 calls `POST /api/intent` with `user_id: "alex_demo"`. The sentence should reference tiramisu because Alex's preferences are in MongoDB. If it's generic, the context assembly function isn't receiving real data.

Phase 2 — E1 + Both Backends Integration (hours 10-12)

E1 removes the mock fallback flags and connects to real backends. In practice, the fallback stays in code — it just stops triggering because the backends are running.

Trigger: E2 says "intent streaming works, confirm returns audio URL."

Test: Full path in browser — tap Food → Dessert → Tiramisu → ElevenLabs voice plays. Caregiver panel shows the session. Knowledge score is 71. Answer the Claude question. Knowledge score ticks to 73.

Integration Checklist

- □ E3: seed script runs, alex_demo profile exists in MongoDB
- □ E3: GET /api/shortcuts returns 5 shortcuts
- □ E3: GET /api/caregiver/panel returns valid panel data
- □ E2: POST /api/intent streams tokens correctly
- □ E2: POST /api/confirm returns audio URL
- □ E2: POST /api/intent with alex_demo references tiramisu in output
- □ E1: Tiramisu path plays audio end-to-end
- □ E1: Caregiver panel shows correct session
- □ E1: Thumbs down + correction saves and clears cache
- □ E1: Knowledge score ticks up when question is answered
- □ Full demo path rehearsed 3 times without touching anything

---

18-HOUR TIMELINE

Hours 0-1: Foundation (All Together)

All three engineers sit together. Write `/shared/api-contract.ts` and `/shared/mock-data.ts` together. Agree on every field name. This takes 30 minutes. Do not rush it — a naming inconsistency discovered at hour 15 costs an hour to fix.

E3: MongoDB Atlas cluster created, connection string in `.env`. Seed script started. E2: FastAPI project scaffold, OpenAI client tested with a single prompt. E1: Next.js project scaffold, Tailwind configured, `api.ts` with mock fallback pattern written.

Hours 1-4: Core Builds

E1: Resizable layout with drag handle. Both columns render. PredictionBar with mock data. TreeGrid with full navigation. Mock intent flow with waveform animation.

E2: `build_context_string()` function complete. `POST /api/intent` streaming with mock user data. Verify first token arrives in under 400ms.

E3: Seed script complete and tested. `GET /api/profile` working. `GET /api/shortcuts` working. Knowledge score engine complete.

Hours 4-8: Feature Builds

E1: OutputPanel with all 5 states. ClarificationButtons component. ShortcutBar with audio prefetch. ComposerGrid with SentenceTray.

E2: `POST /api/confirm` with ElevenLabs synthesis. Both voices working. `POST /api/clarify` generating option cards. Background task for post-session question.

E3: Sentence cache (create + invalidate). Urgency detection. First occurrence detection. `GET /api/caregiver/panel` aggregation. `GET /api/predictions`.

Hours 8-10: E3 + E2 Integration

E2 switches from mock data to real E3 data. Test that sentences reference Alex's known preferences. Fix any data shape mismatches. E1 continues building independently using mocks.

Hours 10-13: E1 + Backend Integration

E1 connects to real backends. Full end-to-end path tested in browser. KnowledgeScore component with breakdown bars. SessionHistory with replay buttons. UrgencyAlert component.

Hours 13-16: Secondary Features

E1: Full caregiver dashboard — all three tabs with real charts. Onboarding flow. Insights tab with Recharts. History tab with PDF export.

E2: Daily digest generation. Voice clone endpoint. `POST /api/demo/seed` trigger.

E3: Session history with filters. `POST /api/sessions/flag`. Custom buttons CRUD. Analytics aggregations for insights charts.

Hours 16-18: Polish + Demo Prep

All engineers: fix bugs found in integration. E1 polish pass — animations, transitions, loading states. Run the full demo path 10 times. Pre-cache

Tiramisu path. Stage the voice contrast moment. Verify urgency alert fires on demand. Register voicemap.tech with LOSALTOS26. Deploy to Vercel.

DEMO SCRIPT

"1 in 3 stroke survivors develops aphasia. Alex is one of them. He understands everything — he just can't form words reliably. Existing communication devices cost $15,000. VoiceMap runs on any laptop, for free."

Beat 1 — Predictions: Open the app. Before Alex taps anything, the prediction bar shows "I need my medication · Good morning · I'm hungry." *"The system already knows it's morning and knows Alex's patterns."*

Beat 2 — Decision tree: Tap Food → Dessert → Tiramisu. Waveform pulses immediately. Sentence streams in. ElevenLabs plays. Label: *"Speaking in Alex's voice."* Right panel: breadcrumb updated live. Session in history with ⭐. *"That's Alex's actual voice, cloned during setup."*

Beat 3 — Composer: Switch to Composer. Tap running icon + sun icon. Confirm. *"I want to go for a morning run."* Plays immediately. *"Two icons, one sentence. For anything the tree can't handle."*

Beat 4 — Learning: Caregiver panel. Thumbs up. Claude's question appears. Type "tiramisu, always tiramisu." Knowledge score ticks from 71% to 73%. Preferences bar fills. *"One question after every session. Every answer improves the next sentence."*

Beat 5 — Voice contrast: Caregiver types "We need to adjust your medication dosage today." Simplify → *"Change your medicine today."* Different voice plays. Label: *"Caregiver voice." "Two voices. Two directions. Alex sounds like himself. His caregiver speaks clearly to him."*

Beat 6 — Dashboard: Open `/caregiver`. Sessions chart shows 30 days of growth. Digest: *"Alex communicated 11 times today. First time he expressed 'I love you' ⭐." "Maya gets this every day. She can share it with his doctor."*

"VoiceMap doesn't just give Alex a voice. It builds a model of who he is — and every session makes that model more accurate. For a population with no affordable tools, that matters."

PRIZE SUBMISSION CHECKLIST

Register voicemap.tech with code `LOSALTOS26` before submitting. 10 minutes, free for 10 years, wins desktop microphones.

Devpost submission structure:

- Opening: social impact framing (aphasia stats, $15k device comparison)
- Technical walkthrough: all four AI behaviors, two ElevenLabs voices, MongoDB knowledge score
- Technical stack section: list every library and service used
- Prize categories to select: ElevenLabs, Best GenAI, MongoDB Atlas, Best Social Impact, Best UI/UX, Most Technically Complex, .Tech Domain

During judging, show in this order:

1. Knowledge score at 71% before any interaction
2. Full demo path with voice playing
3. Knowledge score ticking up after caregiver answers question
4. Open MongoDB Atlas in a browser tab — show alex_demo document live
5. Show the insights charts with 30 days of growth data

VoiceMap — Testing System Plan

Senior QA Engineer Brief · 3-Service Architecture

OVERVIEW

This document describes what the testing system should do, how each test works conceptually, what it measures, and what counts as a pass or failure. It does not contain final production code — it describes the testing architecture so any engineer can implement it.

The testing system has four layers that run independently and in combination:

- **Unit tests** — each service tested in complete isolation
- **Integration tests** — pairs of services tested together
- **End-to-end tests** — full flow from simulated frontend through E2 to E3 and back
- **Performance tests** — latency, throughput, and stress scenarios

All tests produce a structured report. Every failure names the exact component and endpoint suspected. No vague failures allowed.

TESTING INFRASTRUCTURE

Test Runner Architecture

The test system is a standalone Python project in `/tests/`. It has no dependency on the frontend code and imports nothing from E2 or E3

directly. It communicates only via HTTP — exactly like real clients do. This ensures tests catch real integration bugs, not import-level illusions.

```
/tests/
├── runner.py            ← main entry point, runs all suites
├── config.py            ← base URLs, timeouts, test user IDs
├── mock_server.py       ← local mock server E1 uses when backends are down
├── report.py            ← structured output formatter
├── unit/
│   ├── test_e2_context.py     ← test context assembly in isolation
│   ├── test_e2_prompts.py     ← test prompt output quality
│   ├── test_e3_knowledge.py   ← test knowledge score computation
│   ├── test_e3_shortcuts.py   ← test shortcut engine
│   └── test_e3_cache.py       ← test sentence cache logic
├── integration/
│   ├── test_e2_e3.py          ← E2 calling E3, data consistency
│   ├── test_streaming.py      ← SSE stream reconstruction
│   └── test_confirm_flow.py   ← full confirm + synthesis chain
├── e2e/
│   ├── test_scenarios.py      ← the 7 named scenarios
│   └── test_demo_path.py      ← the exact demo path rehearsed
├── performance/
```

```
│   ├── test_latency.py       ← timing measurements
│   └── test_stress.py        ← spam tapping, concurrent requests
└── fixtures/
    ├── seed_test_user.py     ← creates a clean test user before each run
    └── cleanup.py            ← removes test data after each run
```

Configuration

```
# config.py

E2_BASE = "http://localhost:8001"

E3_BASE = "http://localhost:8002"


TEST_USER_ID = "test_user_qa"   # separate from alex_demo, never used in demo

DEMO_USER_ID = "alex_demo"      # used only in demo path tests


LATENCY_THRESHOLDS = {

    "intent_first_token_ms": 500,

    "intent_full_sentence_ms": 1500,

    "confirm_audio_url_ms": 2000,

    "clarify_options_ms": 600,

    "e3_read_ms": 100,
```

```
    "e3_write_ms": 150,
}


STREAMING_FORMAT = {
    "token_key": "token",
    "done_key": "done",
    "session_id_key": "session_id",
    "confidence_key": "confidence",
}
```

Test User Isolation

Every test run starts by creating a fresh `test_user_qa` document in MongoDB via `POST /api/demo/seed` with a test-specific user ID. This user is completely separate from `alex_demo`. Tests never modify `alex_demo`. If a test corrupts state, it only affects `test_user_qa`, which is wiped and re-seeded before the next run.

The cleanup script runs after every test suite regardless of pass/fail. If the test runner crashes, a manual cleanup command is available.

Report Format

Every test produces this exact output structure:

```
═══════════════════════════════════════════════════════════
═══

TEST: [name]
```

═══
═══

Steps:

1. [what was done]
2. [what was done]

...

Observed:

[raw values, timings, response shapes]

Result: PASS ✓ / FAIL ✗ / WARN ⚠

If FAIL:

Suspected component: [E2/E3] [endpoint]

Reason: [specific, not vague]

Raw error: [exact error message or unexpected value]

═══
═══

Warnings are used when behavior is technically correct but concerning — for example, a response that passes but takes 1.8 seconds is not a failure but gets flagged for attention.

UNIT TESTS

E2 Unit Tests

These test E2's internal logic without any network calls. They import E2's service functions directly and call them with controlled inputs.

Test: Context Assembly — Completeness

What it does: Calls `build_context_string()` with a fully populated user object. Verifies the output string contains the patient name, at least one medication, the known preferences, and at least one correction from the correction history.

What it's checking: That no field is silently dropped during assembly.

Pass condition: All five expected substrings present in the output string. **Fail condition:** Any field missing — suspected `build_context_string()` in E2.

Test: Context Assembly — Token Cap

What it does: Calls `build_context_string()` with an artificially large user object — 50 corrections, 100 context answers, a 5000-character preferences string. Counts the tokens in the output using a tokenizer.

What it's checking: The hard cap at 300 tokens is enforced regardless of input size.

Pass condition: Output token count ≤ 300. **Fail condition:** Token count exceeds 300 — suspected context cap logic in E2. This would cause latency to spike in production.

Test: Context Assembly — Priority Order

What it does: Creates a user with data in all five priority tiers. Fills the object until the cap would be hit at tier 3 (so tiers 4 and 5 get dropped).

Verifies that corrections appear in the output but profile basics (lowest priority) do not, or that higher-priority items are present when lower-priority ones are truncated.

What it's checking: When the cap forces truncation, the right things get dropped.

Pass condition: Corrections and context answers present, generic profile fields dropped when cap hit. **Fail condition:** Corrections missing but generic data present — priority order inverted.

Test: Confidence Score Range

What it does: Calls the confidence scoring function with ten different (path, context) combinations ranging from obviously specific ("food→dessert→tiramisu" with a user who loves tiramisu) to completely ambiguous ("feelings"). Collects all ten confidence values.

What it's checking: Confidence scores are always floats between 0.0 and 1.0, and that specific paths score higher than ambiguous ones.

Pass condition: All values in [0.0, 1.0]. Specific path scores ≥ ambiguous path scores. **Fail condition:** Any value outside range — suspected confidence computation in E2. Specific path scores lower than ambiguous — prompt logic inverted.

E3 Unit Tests

These test E3's data logic functions directly without network calls.

Test: Knowledge Score — Baseline

What it does: Creates a minimal user with only a name filled in. Calls the knowledge score computation function. Records the score.

What it's checking: A barely-filled profile scores low but not zero.

Pass condition: Score between 3 and 15. **Fail condition:** Score 0 (computation broken) or > 20 (scoring too generous for minimal data).

Test: Knowledge Score — Incremental Increases

What it does: Starts with the minimal user from the previous test. Adds data one field at a time in the order: medication, preferences text, one context answer, one correction, a second context answer. After each addition, calls the computation function and records the score.

What it's checking: Score increases monotonically as more data is added. No addition causes the score to decrease.

Pass condition: Each successive score ≥ previous score. Final score significantly higher than baseline. **Fail condition:** Any decrease — suspected sub-score computation has a negative contribution somewhere.

Test: Knowledge Score — Category Boundaries

What it does: Adds 20 context answers to the test user (the maximum before the 25-point cap is hit). Then adds a 21st. Calls computation.

What it's checking: The category cap is enforced — adding beyond the maximum doesn't increase the score further.

Pass condition: Score with 20 answers equals score with 21 answers in the conversation sub-score. **Fail condition:** Score keeps increasing past cap — category caps not enforced, score could theoretically exceed 100.

Test: Shortcut Engine — Threshold

What it does: Sets `path_frequencies` for a test user with three paths: one at 3 taps, one at exactly 5 taps (the default threshold), one at 8 taps. Calls the shortcut computation function.

What it's checking: Only paths at or above threshold appear in shortcuts.

Pass condition: Two shortcuts returned (5 and 8 tap paths). The 3-tap path absent. **Fail condition:** All three returned (threshold not enforced) or only one returned (off-by-one error on threshold).

Test: Shortcut Engine — Ordering

What it does: Creates five paths all above threshold with tap counts 5, 7, 12, 9, 6. Calls shortcut engine.

What it's checking: Results are sorted by tap count descending.

Pass condition: Results in order 12, 9, 7, 6, 5. **Fail condition:** Any other order — sort logic broken.

Test: Sentence Cache — Write and Read

What it does: Calls the cache write function with a known path key and sentence. Immediately calls the cache read function with the same path key.

What it's checking: What was written can be read back with identical content.

Pass condition: Read returns exactly what was written. **Fail condition:** Read returns null (write failed silently) or different content (key collision or serialization error).

Test: Sentence Cache — Invalidation

What it does: Writes a sentence to the cache. Calls the invalidation function for the same path key. Calls the cache read function.

What it's checking: Invalidation actually removes the entry.

Pass condition: Read returns null after invalidation. **Fail condition:** Old sentence still returned — invalidation not working, corrections would never take effect.

Test: Urgency Detection — Threshold

What it does: Creates three test sessions with distress paths timestamped within the last 2 hours. Calls the urgency detection function.

What it's checking: Three distress sessions within 2 hours returns `urgent: true`.

Pass condition: Returns `urgent: true`. **Fail condition:** Returns `urgent: false` — urgency alert will never fire during demo.

Test: Urgency Detection — Window

What it does: Creates three distress sessions — two within the last 2 hours, one from 3 hours ago. Calls urgency detection.

What it's checking: Sessions outside the 2-hour window don't count toward the threshold.

Pass condition: Returns `urgent: false` (only 2 within window, below threshold of 3). **Fail condition:** Returns `urgent: true` — time window not applied correctly, false alerts in production.

Test: First Occurrence Detection

What it does: Calls first occurrence check for a path that has never been used by the test user. Records result. Creates a session with that path. Calls first occurrence check again for the same path.

What it's checking: First call returns true, second call returns false.

Pass condition: First check = true, second check = false. **Fail condition:** Either both true (not checking history) or both false (always returning false).

INTEGRATION TESTS

Streaming Tests

The streaming system is the highest-risk technical component. A broken stream looks like a frozen app to the user. These tests are the most important in the integration suite.

Test: Stream Reconstruction — Basic

What it does: Sends a POST request to `/api/intent` with a known path. Reads the response as a raw byte stream. Parses each `data:` line as it arrives. Reconstructs the full sentence by concatenating all token values. Records the timestamp of the first token arrival and the timestamp of the final `done` message.

What it's checking:

- Response begins as SSE format (`Content-Type: text/event-stream`)
- Each line follows the exact format `data: {"token": "..."}`
- Tokens arrive in sequential order with no gaps
- Final message contains `done: true`, `session_id`, and `confidence`
- Reconstructed sentence is coherent English

Pass conditions:

- First token arrives within 500ms of request
- Full sentence reconstructed correctly
- Final message contains all required fields
- `confidence` is a float between 0.0 and 1.0

Fail conditions and suspected components:

- No response within 500ms → E2 `/api/intent` not streaming, or OpenAI connection failed
- Response arrives all at once (not streamed) → streaming not enabled in OpenAI call, or FastAPI not yielding incrementally
- Tokens arrive out of order → SSE event loop ordering bug in E2

- Final message missing `session_id` → E3 session creation failed during stream
- Final message missing `confidence` → confidence computation not completing before stream closes
- Reconstructed sentence is empty → tokens are being sent but `token` key is wrong format

Test: Stream Reconstruction — Token Order Verification

What it does: Reconstructs a stream and assigns each token an index as it arrives. After stream completes, verifies that the sentence built from tokens in arrival order matches the sentence built from tokens sorted by index. Also checks for duplicate tokens (same index appearing twice) and gaps (index 3 followed by index 5 with no index 4).

What it's checking: Tokens are delivered exactly once, in order, with no gaps.

Pass condition: Arrival order equals index order. No duplicates. No gaps. **Fail condition:** Any duplicate → SSE generator has a retry/double-send bug. Any gap → token dropped during transmission, likely a network buffering issue or incorrect chunk boundary handling.

Test: Stream — Premature Termination Detection

What it does: Sends an intent request and forcibly closes the connection after receiving exactly 3 tokens. Waits 5 seconds. Sends the same intent request again normally and completes the full stream.

What it's checking: A dropped connection on one request doesn't corrupt subsequent requests. The second request completes normally.

Pass condition: Second request completes with a full valid stream. No error state persists. **Fail condition:** Second request returns error or partial stream → E2 has connection state leaking between requests. This would cause real user sessions to corrupt each other.

Test: Stream — Concurrent Requests

What it does: Fires 5 simultaneous intent requests with different paths. Reconstructs all 5 streams in parallel. Records the sentence produced by each.

What it's checking: Concurrent streams don't interfere with each other. Each stream produces the correct sentence for its own path.

Pass condition: All 5 streams complete. Each sentence is relevant to its input path. No stream receives tokens from another stream's response. **Fail condition:** Any stream produces a sentence irrelevant to its path → streams are crossing, likely a shared state bug in E2's stream generator.

E2 → E3 Integration Tests

Test: Session Creation During Intent

What it does: Before calling `/api/intent`, records the current session count in E3 for the test user. Calls intent. Waits for stream to complete including the `done` message. Queries E3's session count again.

What it's checking: A pending session is created in E3 during or immediately after the intent stream.

Pass condition: Session count increases by exactly 1. New session has the correct path and sentence. **Fail condition:** Count unchanged → E2 is not calling E3's session creation endpoint. Count increases by more than 1 → duplicate session creation, likely a retry loop in E2.

Test: Context Quality With Real Data

What it does: Queries E3 for `alex_demo`'s full profile. Identifies a distinctive personal detail (favorite food = tiramisu). Calls `/api/intent` with the path `["food","dessert"]` — an ambiguous path that without

personalization would produce a generic sentence. Checks if the response mentions tiramisu.

What it's checking: E2 is successfully reading Alex's profile from E3 and using it in the prompt, not falling back to mock data or generic context.

Pass condition: Sentence contains "tiramisu" or clearly references a specific preference. **Fail condition:** Sentence is generic ("I would like dessert") → E2 not receiving E3 data, or context assembly function not including preferences correctly.

Test: Correction Learning Loop

What it does: Calls `/api/intent` with path `["food","dessert","cake"]`. Records the sentence. Calls `/api/feedback` with `thumbs_up: false` and correction `"I want tiramisu specifically, not cake"`. Waits 2 seconds for background processing. Calls `/api/intent` with the same path again.

What it's checking: After a correction, the next generation for that path produces a sentence influenced by the correction.

Pass condition: Second sentence references tiramisu or is noticeably different from and more specific than the first. **Fail condition:** Second sentence identical to first → correction not saved to E3, or E2 not reading correction history, or sentence cache not invalidated.

Test: Audio URL Validity

What it does: Calls `/api/confirm` with a valid session ID. Records the returned `audio_url`. Makes an HTTP GET request to that URL.

What it's checking: The audio URL is a real, accessible file.

Pass condition: GET request to audio URL returns 200 with `Content-Type: audio/mpeg` or similar. File size > 0 bytes. **Fail condition:** 404 → ElevenLabs synthesis succeeded but file not saved to

expected path. 200 but empty file → synthesis failed silently. URL is a mock URL that was never replaced with a real one → E2 returning mock data in production.

END-TO-END SCENARIO TESTS

These are the 7 named scenarios from the brief. Each is described as a complete test with exact steps, measurements, and failure attribution.

Scenario 1: Basic Happy Path

Purpose: Verify the core communication flow works completely end-to-end.

Steps:

1. Seed test user with `alex_demo` preferences (loves tiramisu)
2. Send `POST /api/intent` with `path: ["food","dessert","tiramisu"]`, `user_id: test_user_qa`
3. Reconstruct full stream, record first token time and full completion time
4. Capture `session_id` and `confidence` from done message
5. Send `POST /api/confirm` with the session_id, record response time
6. Capture `audio_url` from confirm response
7. Verify session exists in E3 via `GET /api/sessions/history`
8. Verify `audio_url` is accessible
9. Verify path frequency incremented in E3 user document

Pass conditions:

- First token < 500ms
- Sentence contains "tiramisu"
- Confidence > 0.7

- Confirm response < 2000ms
- Audio URL returns 200
- Session exists in E3 with correct path
- Path frequency for `food→dessert→tiramisu` is 1 higher than before test

Fail attribution:

- Slow first token → E2 `/api/intent` OpenAI latency
- Sentence doesn't mention tiramisu → E2 context assembly not using E3 data
- Confirm slow → E2 ElevenLabs synthesis timeout
- Session missing in E3 → E2 not calling E3 session create, or E3 session create broken
- Path frequency not updated → E3 `sessions/confirm` not incrementing frequencies

---

Scenario 2: Fast Repeated Tapping (Spam Simulation)

Purpose: Verify the system handles rapid repeated requests without crashing, corrupting data, or producing inconsistent responses.

Steps:

1. Fire 10 simultaneous `POST /api/intent` requests with identical path `["food","dessert","tiramisu"]` and same user_id
2. Reconstruct all 10 streams in parallel
3. Record all 10 sentences and confidence scores
4. After all complete, count sessions created in E3 for this path
5. Check for any 500 errors or malformed responses among the 10

Pass conditions:

- All 10 requests return valid streams (no 500s)
- All 10 sentences are coherent and relevant to path

- Session count in E3 increases by ≤ 10 (ideally exactly 10, but cache hits are acceptable)
- No duplicate session IDs across the 10 responses
- Confidence scores are consistent (within ±0.15 of each other — same path should produce similar confidence)

Fail conditions:

- Any 500 error → E2 not handling concurrent requests, likely a shared state or connection pool issue
- Fewer than 10 sessions created → session creation has a race condition dropping writes
- Duplicate session IDs → UUID generation not random enough or session IDs being shared between requests
- Wildly inconsistent confidence scores → confidence computation is non-deterministic in a problematic way

---

Scenario 3: Long Path Stress Test

Purpose: Verify the system handles unusually deep or complex paths without breaking stream format or producing errors.

Steps:

1. Construct a long path: `["health","needs","pain","urgent","left","arm","after","fall","yesterday","morning"]`
2. Send `POST /api/intent` with this path
3. Reconstruct full stream
4. Verify sentence is coherent English
5. Verify JSON format of done message is valid
6. Verify confidence score is present and in range

Pass conditions:

- Stream completes without error
- Sentence is coherent (describes pain or injury in some way)
- Done message is valid JSON with all required fields
- No JSON parse errors during stream reconstruction

Fail conditions:

- Stream stops mid-way → E2 has a timeout triggered by long prompts
- Done message is malformed JSON → long path caused string escaping issue in E2's JSON construction
- Sentence is empty or "undefined" → path-to-prompt conversion fails on long inputs
- 500 error → E2 crashes on input above an expected length

Scenario 4: Invalid Input Handling

Purpose: Verify the system fails gracefully on bad inputs without crashing or exposing internal errors.

Sub-test A — Empty path:

Send `POST /api/intent` with `path: []`.

Expected: Either a graceful error response with a human-readable message, or a fallback sentence like "I need help" with low confidence. Not acceptable: 500 server error, null response, or a crash.

Sub-test B — Null path:

Send `POST /api/intent` with `path: null`.

Expected: Same as above — graceful handling. Not acceptable: Unhandled exception, stack trace in response.

Sub-test C — Invalid user_id:

Send `POST /api/intent` with `user_id: "definitely_not_a_real_user_xyz"`.

Expected: Either a 404 with clear message, or a fallback sentence generated without personalization. Not acceptable: 500 error, empty response, or a sentence that claims to be personalized but isn't.

Sub-test D — Path with special characters:

Send `POST /api/intent` with `path: ["food", "dessert", "<script>alert('xss')</script>"]`.

Expected: Graceful handling — either sanitized path or error response. Not acceptable: XSS content reflected in sentence, 500 error.

Fail attribution for all sub-tests:

- 500 error → Input validation missing in E2 or E3
- Stack trace in response → Error handling not catching exceptions in E2
- Crash requiring server restart → Unhandled exception at process level

---

Scenario 5: Confidence Routing and Clarification

Purpose: Verify that low-confidence responses correctly trigger the clarification flow and return properly structured options.

Steps:

1. Identify or construct a path known to produce low confidence (single-element ambiguous path like `["feelings"]`)
2. Send `POST /api/intent`. Reconstruct stream. Record confidence.
3. If confidence < 0.5: send `POST /api/clarify` with same path
4. Verify response structure of clarify endpoint

5. For each option in response, verify it has `label`, `icon`, and `path` fields
6. For each option's path, verify it would produce a higher-confidence sentence by calling intent again

Pass conditions:

- Ambiguous path produces confidence < 0.5
- Clarify endpoint returns 2-3 options
- Each option has `label` (string), `icon` (valid Phosphor icon name string), `path` (array of strings)
- Calling intent with each option's path produces confidence > 0.7

Fail conditions:

- Ambiguous path produces confidence > 0.7 → confidence scoring not working, all paths appear confident
- Clarify returns only 1 option → not enough branching in prompt
- Options missing `icon` field → JSON schema in E2 prompt missing icon requirement
- Option paths produce same low confidence as original → clarification options are as ambiguous as the original input

---

Scenario 6: Data Persistence Verification

Purpose: Verify that a complete intent + confirm cycle correctly persists all expected data to E3.

Steps:

1. Record baseline state: session count, path frequency for test path, shortcut list
2. Call `/api/intent` with path `["food","dessert","tiramisu"]`
3. Complete stream, capture session_id

4. Call `/api/confirm` with session_id
5. Wait 500ms for async operations to complete
6. Query E3 for: session document, path frequency, shortcut list, audio_url on session
7. Call `/api/context/answer` with a test question and answer
8. Query E3 for: updated knowledge score, context_log entry, updated context_answers

Pass conditions:

- Session document exists with correct path, sentence, and audio_url
- Path frequency for `food→dessert→tiramisu` incremented by 1
- If frequency now ≥ threshold, path appears in shortcuts list
- Knowledge score higher after context answer
- Context log contains the question and answer
- User document's context_answers array contains the new answer

Fail conditions:

- Session missing or has null audio_url → confirm not completing E3 write
- Path frequency unchanged → session confirm not calling frequency increment
- Knowledge score unchanged after context answer → `/api/context/answer` not triggering knowledge computation
- Context log missing entry → E3 context answer save function failing silently

---

Scenario 7: Performance Benchmarks

Purpose: Measure real latency numbers across all critical endpoints and flag anything exceeding thresholds.

What to measure for each endpoint:

For `/api/intent`: time from request sent to first token received (first token latency), time from request sent to done message received (full completion latency). Run 5 times and record min, max, and median.

For `/api/confirm`: time from request sent to response received with audio_url. Run 5 times.

For `/api/caregiver/panel`: time from request sent to full JSON response. This is polled every 3 seconds so it must be very fast. Run 10 times.

For `/api/shortcuts`: same as panel. Run 10 times.

For `/api/clarify`: time from request to full response. Run 5 times.

Thresholds (from config):

Endpoint	Warning	Failure
intent first token	> 400ms	> 500ms
intent full sentence	> 1000ms	> 1500ms
confirm audio_url	> 1500ms	> 2000ms

caregiver panel	> 200ms	> 500ms
shortcuts	> 100ms	> 300ms
clarify	> 400ms	> 600ms

Additional measurements:

- Memory usage of E2 and E3 processes before and after 50 requests (check for memory leaks)
- MongoDB query time for panel aggregation (should be under 50ms — add logging in E3)
- Cache hit rate after 10 repeated requests to same path (should be 90%+ after first request)

Report format for performance tests:

PERFORMANCE REPORT

═══════════════════════

/api/intent (first token)

Median: 342ms ✓

Min: 287ms

Max: 891ms ⚠ (single spike — check if OpenAI had momentary delay)

/api/confirm

Median: 1,847ms ⚠ (approaching 2s threshold)

Min: 1,203ms

Max: 2,341ms ✗ (exceeded 2s threshold — ElevenLabs synthesis slow)

Recommendation: Switch to eleven_turbo_v2 for lower latency

EDGE CASE TESTS

Beyond the 7 scenarios, these additional edge cases should be tested:

Cache Coherence After Correction

What it tests: After a caregiver correction invalidates a cached sentence, the next request regenerates fresh rather than serving stale cache.

Steps: Generate and cache a sentence. Submit a correction. Immediately request the same path. Verify the new sentence incorporates the correction and does not match the pre-correction cached sentence.

Why it matters: If cache invalidation fails, the correction learning loop is silently broken — corrections appear to save but the patient hears the wrong sentence forever.

Knowledge Score Race Condition

What it tests: Two simultaneous profile updates don't corrupt the knowledge score.

Steps: Fire two simultaneous `POST /api/profile/update` requests with different fields. Wait for both to complete. Query the knowledge score. Verify the score reflects both updates.

Why it matters: The caregiver dashboard allows inline editing of multiple fields. If two fields are blurred in quick succession and both trigger score updates simultaneously, the second update might overwrite the first without including it.

Session History Pagination

What it tests: Sessions beyond the first page are correctly returned.

Steps: Seed 25 sessions for test user. Request history with `limit: 10`. Verify 10 returned. Note the oldest timestamp. Request history again with a date range that excludes the first 10. Verify next 10 returned with no overlap.

Why it matters: The caregiver history tab filters by date range. Broken pagination means caregivers can't review older sessions.

First Occurrence — Concurrent Sessions

What it tests: If two sessions with the same new path are created simultaneously, only one gets `is_first_occurrence: true`.

Steps: Fire two simultaneous confirm requests for sessions with a path that has never been used. Check both resulting session documents.

Why it matters: If both get flagged as first occurrences, the ⭐ badge loses meaning. The first occurrence check has a race condition risk.

Urgency Alert — Dismiss and Re-trigger

What it tests: After an urgency alert is dismissed, it doesn't immediately re-trigger, and it does correctly re-trigger after the window resets.

Steps: Trigger urgency condition (3 distress sessions in 2 hours). Verify `urgent: true` in panel response. Call dismiss endpoint. Verify `urgent: false` in next panel response. Add a 4th distress session. Verify `urgent: true` again.

Why it matters: If dismiss doesn't actually suppress the alert, the caregiver can never clear it.

DEMO PATH TESTS

These tests run specifically against `alex_demo` (not the test user) and verify the exact demo sequence works perfectly. They should be run immediately before the hackathon presentation.

Full Demo Rehearsal Test

Steps:

1. Verify `alex_demo` profile exists and has correct data
2. Call predictions for current hour — verify "I need my medication" appears
3. Run full Tiramisu path intent + confirm — verify audio URL accessible
4. Verify localStorage fallback key exists after confirm
5. Run icon composer path `["running","sun"]` — verify sentence references morning run
6. Submit a thumbs up — verify session feedback saved
7. Verify Claude question appears in panel response
8. Answer the question — verify knowledge score increases
9. Verify caregiver simplify produces two distinct voices (different voice IDs)
10. Verify urgency alert can be manually triggered (set 3 distress sessions to current timestamp)

Pass condition: All 10 steps pass with no failures. Total time under 45 seconds. **Fail condition:** Any step fails → fix before presenting. This test is the final check before demo.

localStorage Fallback Test

Steps: Run the Tiramisu path once (populates localStorage). Stop E2 process. Run the Tiramisu path again via the frontend (or simulate the frontend's fetch logic). Verify the cached sentence and audio are returned without hitting the backend.

Why it matters: This is your safety net if hackathon wifi dies during the demo.

HOW TO RUN

```
# Full test suite

python tests/runner.py --all


# Individual suites

python tests/runner.py --unit

python tests/runner.py --integration

python tests/runner.py --e2e

python tests/runner.py --performance


# Pre-demo check only
```

```
python tests/runner.py --demo

# With verbose output
python tests/runner.py --all --verbose

# Reset test data only
python tests/fixtures/cleanup.py
python tests/fixtures/seed_test_user.py
```

The runner prints a summary at the end:

```
═══════════════════════════════════════════════════════════
VOICEMAP TEST SUITE — SUMMARY
═══════════════════════════════════════════════════════════
Unit tests:         23 pass  0 fail  1 warn
Integration tests:  12 pass  1 fail  0 warn
E2E scenarios:       7 pass  0 fail  0 warn
Performance:         5 pass  0 fail  2 warn
Demo path:          10 pass  0 fail  0 warn
───────────────────────────────────────────────────────────
TOTAL: 57 pass, 1 fail, 3 warn
```

FAILURES:

✗ integration/test_e2_e3.py::test_correction_learning_loop

Suspected: E2 /api/feedback — correction not invalidating cache

Raw: Second sentence identical to first after correction submitted

WARNINGS:

⚠ performance::confirm_latency — median 1,847ms (threshold 2,000ms)

⚠ performance::intent_max — single spike at 891ms (threshold 500ms first token)

⚠ unit::context_assembly — 287 tokens (approaching 300 cap)

One glance tells you exactly what's broken, where it is, and what to fix before the demo.